

Exp no: 10**Troubleshooting Fabric Applications****Aim**

To identify, analyze, and resolve common issues encountered in Hyperledger Fabric applications by troubleshooting network connectivity, chaincode deployment, channel configuration, certificate authentication, and Docker container errors

Step 1: Navigate to the Fabric Test Network

None

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

First make sure any previous experiment is stopped:

None

```
./network.sh down
```

Then start a fresh network:

None

```
./network.sh up createChannel -ca
```


Step 4: Check Docker Containers

None

docker ps

Step 5: Check Peer Logs

CONTAINER ID	IMAGE	PORTS	NAMES	COMMAND	CREATED	STATUS
661155b28810	dev-peer0.org2.example.com-basic_1.0-1d4d45573112813889f87c307bc2b5f8e664c87b24c835bee15cbcc7f59b629-c68f8ecc2a8ff8cc1497615f5e33a8bfa988e8fd3555f864cf1398b52d341cb		dev-peer0.org2.example.com-basic_1.0-1d4d45573112813889f87c307bc2b5f8e664c87b24c835bee1	"docker-entrypoint.s..."	6 minutes ago	Up 6 minute
8dc6cf59b029	dev-peer0.org1.example.com-basic_1.0-1d4d45573112813889f87c307bc2b5f8e664c87b24c835bee15cbcc7f59b629-d1cf9b1f4b9747cc843ac8e74bfaeade1cf0b07f5c1163835f604aa885d60dc		dev-peer0.org1.example.com-basic_1.0-1d4d45573112813889f87c307bc2b5f8e664c87b24c835bee1	"docker-entrypoint.s..."	6 minutes ago	Up 6 minute
8dc6cf59b029	hyperledger/fabric-orderer:latest		orderer.example.com	"orderer"	9 minutes ago	Up 9 minute
88e0ff42d3609	hyperledger/fabric-peer:latest	0.0.0.0:7050→7050/tcp, [::]:7050→7050/tcp, 0.0.0.0:7053→7053/tcp, [::]:7053→7053/tcp, 0.0.0.0:9443→9443/tcp, [::]:9443→9443/tcp	peer0.org1.example.com	"peer node start"	9 minutes ago	Up 9 minute
89f944060f5a	hyperledger/fabric-peer:latest	0.0.0.0:7051→7051/tcp, [::]:7051→7051/tcp, 0.0.0.0:9444→9444/tcp, [::]:9444→9444/tcp	peer0.org2.example.com	"sh -c 'fabric-ca-se..."	9 minutes ago	Up 9 minute
7053d2596eb8	hyperledger/fabric-ca:latest	0.0.0.0:9051→9051/tcp, [::]:9051→9051/tcp, 7051/tcp, 0.0.0.0:9445→9445/tcp, [::]:9445→9445/tcp	ca_orderer	"sh -c 'fabric-ca-se..."	9 minutes ago	Up 9 minute
02b6a52c0f5f	hyperledger/fabric-ca:latest	0.0.0.0:9054→9054/tcp, [::]:9054→9054/tcp, 7054/tcp, 0.0.0.0:18054→18054/tcp, [::]:18054→18054/tcp	ca_org2	"sh -c 'fabric-ca-se..."	9 minutes ago	Up 9 minute
99526fc8dfd3	hyperledger/fabric-ca:latest	0.0.0.0:7054→7054/tcp, [::]:7054→7054/tcp, 0.0.0.0:17054→17054/tcp, [::]:17054→17054/tcp	ca_org1	"sh -c 'fabric-ca-se..."	9 minutes ago	Up 9 minute
3c23d975a119	ghcr.io/eramba/eramba:latest	80/tcp	cron	"/docker-cron-entryp..."	3 weeks ago	Up 4 hours
6e575645bc7e	ghcr.io/eramba/eramba-mcp-server:latest	3000/tcp	mcp_server	"/docker-entrypoint.s..."	3 weeks ago	Up 4 hours
496f6dc04b0	ghcr.io/eramba/eramba:latest	80/tcp	eramba	"docker-entrypoint.s..."	3 weeks ago	Up 4 hours
03e16bcef0b0	redis:7.4.2-alpine		redis	"docker-entrypoint.s..."	3 weeks ago	Up 4 hours
4a98fac37466	mysql:8.4.3-oracle		mysql	"docker-entrypoint.s..."	3 weeks ago	Up 4 hours

Step 5: Check Peer Logs

None

docker logs peer0.org1.example.com

If there is too much output, use:

None

docker logs --tail 10 peer0.org1.example.com

```
root@ram: ~/fabric-dev/fabric-samples/test-network
$ docker logs --tail 10 peer0.org1.example.com
2026-09-08 13:17:03.618 UTC 067d INFO [endorser] callChaincode → finished chaincode: basic duration: 9ms channel=mychannel txID=6e8f8779
2026-09-08 13:17:03.411 UTC 007e INFO [comm.grpc.server] 1 → unary call completed grpc.service=protos.Endorser grpc.method=ProcessProposal grpc.peer_address=172.18.0.1:49710 grpc.code=OK grpc.call_duration=10.59141ms
2026-09-08 13:17:03.425 UTC 007f INFO [gosip.privdata] StoreBlock → Received block [6] from buffer channel=mychannel
2026-09-08 13:17:05.428 UTC 0080 INFO [committer.txvalidator] Validate → [mychannel] Validated block [6] in 2ms
2026-09-08 13:17:05.431 UTC 0081 INFO [kvledger] commit → [mychannel] Committed block [6] with 1 transaction(s) in 3ms (state_validation=0ms block_and_privdata_commit=1ms state_commit=0ms) commitHash=[f671e377cd13170de7eccdb8e2537d8e2658498ac04801a36c30609004a607]
2026-09-08 13:17:05.434 UTC 0082 INFO [comm.grpc.server] 1 → streaming call completed grpc.service=protos.Deliver grpc.method=DeliverFiltered grpc.request_deadline=2026-09-08T13:17:33.415Z grpc.peer_address=172.18.0.1:49712 error="context finished before block retrieved: context canceled" grpc.code=Unknown grpc.call_duration=2.018543364s
2026-09-08 13:17:11.118 UTC 0083 INFO [endorser] callChaincode → finished chaincode: basic duration: 5ms channel=mychannel txID=82b433ab
2026-09-08 13:17:11.118 UTC 0084 INFO [comm.grpc.server] 1 → unary call completed grpc.service=protos.Endorser grpc.method=ProcessProposal grpc.peer_address=172.18.0.1:43274 grpc.code=OK grpc.call_duration=6.349912ms
2026-09-08 13:17:45.809 UTC 0085 INFO [endorser] callChaincode → finished chaincode: basic duration: 12ms channel=mychannel txID=54b07653
2026-09-08 13:17:45.809 UTC 0086 INFO [comm.grpc.server] 1 → unary call completed grpc.service=protos.Endorser grpc.method=ProcessProposal grpc.peer_address=172.18.0.1:51994 grpc.code=OK grpc.call_duration=14.621643ms
```

Step 6: Check Orderer Logs

None

docker logs orderer.example.com

Or:

None

```
docker logs --tail 50 orderer.example.com
```

Step 7: Check Certificate Authority

None

```
docker logs ca_org1
```

Step 8: Verify Channel Membership

None

```
peer channel list
```

Step 9: Deploy Chaincode

None

```
./network.sh deployCC \  
-ccl javascript \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript
```

```
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]  
Query chaincode definition successful on peer0.org1 on channel 'mychannel'  
Using organization 2  
Querying chaincode definition on peer0.org2 on channel 'mychannel'...  
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.  
peer lifecycle chaincode querycommitted --channelID mychannel --name basic  
res=0  
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':  
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]  
Query chaincode definition successful on peer0.org2 on channel 'mychannel'  
Chaincode initialization is not required
```

Step 12: Test the Chaincode

Query the ledger:

```
peer chaincode query \
```

Step 10: Verify Chaincode Installation

None

```
peer lifecycle chaincode queryinstalled
```

```
(ram@ram)-[~/fabric-dev/fabric-samples/test-network]
$ peer lifecycle chaincode queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15cbcc7f59b629, Label: basic_1.0
```

Step 11: Verify Chaincode on the Channel

None

```
peer lifecycle chaincode querycommitted \
-C mychannel \
-n basic
```

```
(ram@ram)-[~/fabric-dev/fabric-samples/test-network]
$ peer lifecycle chaincode querycommitted \
-C mychannel \
-n basic
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
```

You should see information such as:

Step 12: Test the Chaincode

Query the ledger:

None

```
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
```

If the ledger has been initialized, you should receive the asset records.

If it returns:

None

```
[]
```

initialize the ledger:

None

```
peer chaincode invoke \
-o localhost:7050 \
```

```

--ordererTLSTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem" \
--waitForEvent

```

Then query again:

```

None

peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'

```

```

[~]$ (cmd) run -f/fabric-dev/fabric-samples/test-network
[~]$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":15,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":15,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]

```

Step 13: Test Reading an Asset

```

None

peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset1"]}'

```

```
(ram@ram)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query \
  C mychannel \
  n basic \
  c '{"Args":["ReadAsset","asset1"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}
```

Step 14: Check Docker Resource Usage

None

`docker stats --no-stream`

```
(ram@ram)-[~/fabric-dev/fabric-samples/test-network]
$ docker stats --no-stream
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
661155b28810	dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbee664c87b24c035bee15cbcc7f59b629	0.00%	54.11MiB / 2GiB	2.64%	342kB / 35.5kB	0B / 4.1kB	24
079171243a88	dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbee664c87b24c035bee15cbcc7f59b629	0.00%	51.77MiB / 2GiB	2.53%	348kB / 38.4kB	0B / 4.1kB	24
08edfd2d3609	orderer.example.com	0.12%	23.48MiB / 15.32GiB	0.15%	95.1kB / 274kB	0B / 287kB	17
01f94a00df5a	peer0.org1.example.com	2.98%	59.8MiB / 15.32GiB	0.38%	737kB / 622kB	0B / 561kB	17
0853d2596ab8	peer0.org2.example.com	2.58%	60.91MiB / 15.32GiB	0.39%	703kB / 575kB	0B / 561kB	17
02b6a52c0f5f	ca_orderer	0.00%	15.88MiB / 15.32GiB	0.10%	52.7kB / 84.8kB	0B / 1.05MB	16
0dae2dc0be7e	ca_org2	0.00%	13.06MiB / 15.32GiB	0.08%	29.5kB / 43.9kB	0B / 737kB	16
09526fc0df03	ca_org1	0.00%	13MiB / 15.32GiB	0.08%	32kB / 50kB	0B / 737kB	15
0c3d6978a119	cron	3.98%	119.4MiB / 15.32GiB	0.76%	22.1MB / 40.3MB	77MB / 2.4MB	7
0e575645bc7e	mcp_server	0.00%	112.1MiB / 15.32GiB	0.71%	4.96kB / 252B	85.1MB / 0B	7
0496f6dc84bd	eramba	0.01%	93.16MiB / 15.32GiB	0.59%	254kB / 12.3kB	107MB / 4.1kB	6
03e16bcef0b0	redis	1.50%	22.83MiB / 15.32GiB	0.14%	80.6kB / 1.98MB	13.7MB / 0B	6
0a86fac37466	mysql	1.58%	584MiB / 15.32GiB	3.72%	40.2MB / 20.3MB	166MB / 866MB	40

Step 15: Check Recent Docker Events

None

`docker events --since 10m`

You may see events such as:

None

Container Started

Network Created

Container Stopped